

Supervised Learning Algorithms

Task 2

Objective

You will implement, train, and evaluate models for both regression and classification tasks. This will involve data preprocessing, model building, performance evaluation, and interpretation of results.

- **Develop Mathematical Intuition** this task isn't just about calling `.fit()` and `.predict()`. Before implementing each algorithm, make sure you properly understand the mathematical foundation behind it its **loss/cost function**, how it's optimized (**gradient descent**, a **closed-form solution like OLS**, or another method depending on the algorithm), what its coefficients/parameters actually represent, and why the chosen evaluation metrics make sense for that algorithm. Superficial, copy-paste usage without this understanding will not be considered complete.

Attribute	Details
Programming Language	Python
Libraries	Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn

Dataset

Use the provided Global Tech Job Market dataset. It is not pre-cleaned, so expect messy values (inconsistent boolean formats, placeholder/missing salary values, etc.) that you'll need to handle during preprocessing.

A Note on Insights

Wherever it's relevant after an EDA step, a chart, a model result, a confusion matrix, or the final comparison add a short **Markdown note above or below that cell** explaining what you're seeing and why it matters. Write this in **your own words**, based on your own understanding of the output. Do not copy-paste explanations from ChatGPT, Gemini, or any other AI tool this will be checked, and generic AI-sounding explanations that don't reflect the actual output in front of you will be marked down. A short, genuine, slightly imperfect insight in your own language is worth far more here than a polished but generic one.

Case 1: Predicting Salary Regression

In this section, you'll build a model to predict a continuous outcome.

- **Objective:** Implement a Linear Regression model to predict a candidate's expected salary based on job posting attributes.
- **Target column:** Not provided you must decide and justify which column(s) to use or derive as your prediction target. Clearly explain your reasoning in a Markdown cell, including how you handled missing/placeholder salary values.

Tasks:

- **Load and Explore:** Load the dataset and perform an exploratory data analysis (EDA). Check for missing values and understand the features.
- **Data Preprocessing:** Clean and standardize messy salary values, encode categorical variables appropriately, and split the data into training (80%) and testing (20%) sets.
- **Model Training:** Instantiate and train a LinearRegression model from Scikit-learn on the training data. In a Markdown cell, briefly note that the model minimizes the **Mean Squared Error cost function**, and that Scikit-learn's LinearRegression solves for the coefficients directly using **OLS (Ordinary Least Squares)** a closed-form solution rather than iterative gradient descent. Briefly note in your own words how this differs from gradient-descent-based optimization used elsewhere in this task.
- **Prediction & Evaluation:** Make predictions on the test set and evaluate your model's performance using:
 - **MAE** (Mean Absolute Error)
 - **MSE** (Mean Squared Error) and **RMSE** (Root Mean Squared Error)
 - **R Score** (Coefficient of Determination)
 - A plot of **Actual vs Predicted** values to visually assess fit

Case 2: Predicting Layoffs Classification

Here you'll tackle a binary classification problem, exploring it through three different model families.

- **Objective:** Predict whether a job posting is associated with a layoff within 12 months.
- **Target column:** Not provided you must decide and justify your target column (hint: look closely at the layoff-related field). This field appears in multiple inconsistent formats across rows clean and standardize it into a single consistent binary column before modeling, and explain how you did this in a Markdown cell.

Shared preprocessing (do this once, use for all three models below):

- **Load and Prepare:** Load the dataset, clean/standardize your target column, handle missing values in relevant feature columns, and encode categorical features.
- **Split:** Split into training (80%) and testing (20%) sets, using stratify on the target to preserve class balance.
- **Feature Scaling:** Apply feature scaling (e.g. StandardScaler) to the features. Fit the scaler only on the training data, then transform both train and test sets.
- **Class Balance Check:** Report the class distribution of your target column note if it's imbalanced, since that affects how you interpret accuracy.

Part 2A Logistic Regression

- Train a LogisticRegression model on the scaled training data.
- In a Markdown cell, briefly note the **sigmoid function** it uses to map outputs to probabilities, and the **log-loss (binary cross-entropy)** cost function that gradient descent minimizes during training and how this differs from the OLS-based cost function used in Case 1.
- Evaluate on the test set: report **accuracy, precision, recall, and F1-score**.
- Generate and display a **confusion matrix**.

Part 2B Decision Tree

- Train a DecisionTreeClassifier model on the same data.
- In a Markdown cell, briefly note the **splitting criterion** the tree uses (Gini impurity or entropy/information gain) to decide each split note that this is a fundamentally different optimization approach from the gradient-descent/OLS-based models above.
- Report the model's **accuracy, precision, recall, and F1-score** on the test set.
- Generate and display a **confusion matrix**.

Part 2C Support Vector Machines (SVM)

- Train two SVM classifiers on the scaled data:
 - one with a linear kernel (kernel='linear')
 - one with a Radial Basis Function kernel (kernel='rbf')
- In a Markdown cell, briefly note the idea of **margin maximization** and **hinge loss** that SVMs optimize, and in your own words, what the **kernel trick** (specifically the RBF kernel) is doing differently from the linear kernel.
- Evaluate and report **accuracy, precision, recall, and F1-score** for both kernel variants.
- Generate and display a **confusion matrix** for each.

Part 2D Comparison

- Present a single summary table comparing all four classifiers built above (Logistic Regression, Decision Tree, SVM-linear, SVM-RBF) across **accuracy, precision, recall, and F1-score**.
- In a Markdown cell, state which model performed best, which metric mattered most for this problem (and why think about what a false negative means here), and offer a brief, reasoned explanation of your choice.